

# Network Science

---

## Lecture 02: Graph Theory

Prof. Dr. Michael Granitzer Dr. Kanishika Ghosh Dastidar

Dr. Jelena Mitrovic

# From Networks to Graphs

Last time we saw that networks are everywhere: social, informational, economic, technical. Today we build the formal language to describe them precisely.

## Today

- what properties do networks have?
- how do we formalize networks as graphs?
- paths and distance
- breadth-first search
- connectivity and components

# Module 1

---

## Properties of Networks

**Guiding Question:** What properties does a network have? What do we need to describe it?

# What Does a Node Mean?

- In a social network: a **person**
- In the Web: a **page**
- In a financial network: an **institution**
- In a transport network: a **city** or **station**

A node is whatever entity you choose to model. That choice is never neutral — it determines what you can see and what you miss.

# What Does an Edge Mean?

- Friendship: **symmetric** — if A is friends with B, B is friends with A
- Following on Twitter/X: **asymmetric** — A follows B does not mean B follows A
- Email: **directed** — sender and receiver are distinct roles
- Co-authorship: **symmetric** — both contributed to the same paper

An edge encodes a relationship. But relationships differ:

- symmetric vs. directed
- strong vs. weak
- binary vs. weighted.

# What About Time?

- Networks **grow**: new people join, new links form
- Networks **shrink**: people leave, links break
- Networks **change**: relationships strengthen or weaken
- The **order** of events matters: who connected to whom first?

Most real networks are dynamic. A static graph is a snapshot — useful, but incomplete.

# From Properties to Formal Language

We now have a list of properties we need to capture:

| Property       | Formal concept                 |
|----------------|--------------------------------|
| Entities       | Nodes (vertices)               |
| Relationships  | Edges (links)                  |
| Directionality | Directed vs. undirected graphs |
| Strength       | Weighted vs. unweighted graphs |
| Structure      | Adjacency matrix, degree       |
| Time           | Temporal / dynamic graphs      |

# Takeaway

Describing a network precisely requires answering:

- What are the nodes? What are the edges?
- Are they directed? Weighted?
- Changing over time?

Graph theory gives us a formal language to answer each of these questions.



# Mini-Exercise 2.A.1

Pick a real-world network you interact with (e.g. your university's course system, a messaging group, a transport network). Write down:

1. What are the **nodes**?
2. What are the **edges**?
3. Is the relation **directed** or **undirected**?
4. Should edges be **weighted**? If so, what does the weight represent?
5. Does the network **change over time**?

# Mini-Exercise 2.A.1 — Sample Answer

## Example: University course enrollment system

1. **Nodes:** Students and courses (two types → bipartite)
2. **Edges:** "Student X is enrolled in course Y"
3. **Direction:** Directed — enrollment is an action from student to course
4. **Weights:** Could represent credit hours, or grade achieved
5. **Time:** Yes — students enroll and drop courses each semester

This is already a bipartite graph with temporal dynamics.

# Module 2

---

## Graph Models

**Guiding Question:** When is a graph model good enough — and what gets lost?

# Why Graph Theory

We identified properties we need to describe. Now we need precise definitions. Graph theory gives us that language and lets us move from intuition to analysis.

# Formal Definition

A graph  $G = (V, E)$  consists of:

- a set of **vertices** (nodes)  $V$
- a set of **edges** (links)  $E \subseteq V \times V$

# Formal Definition

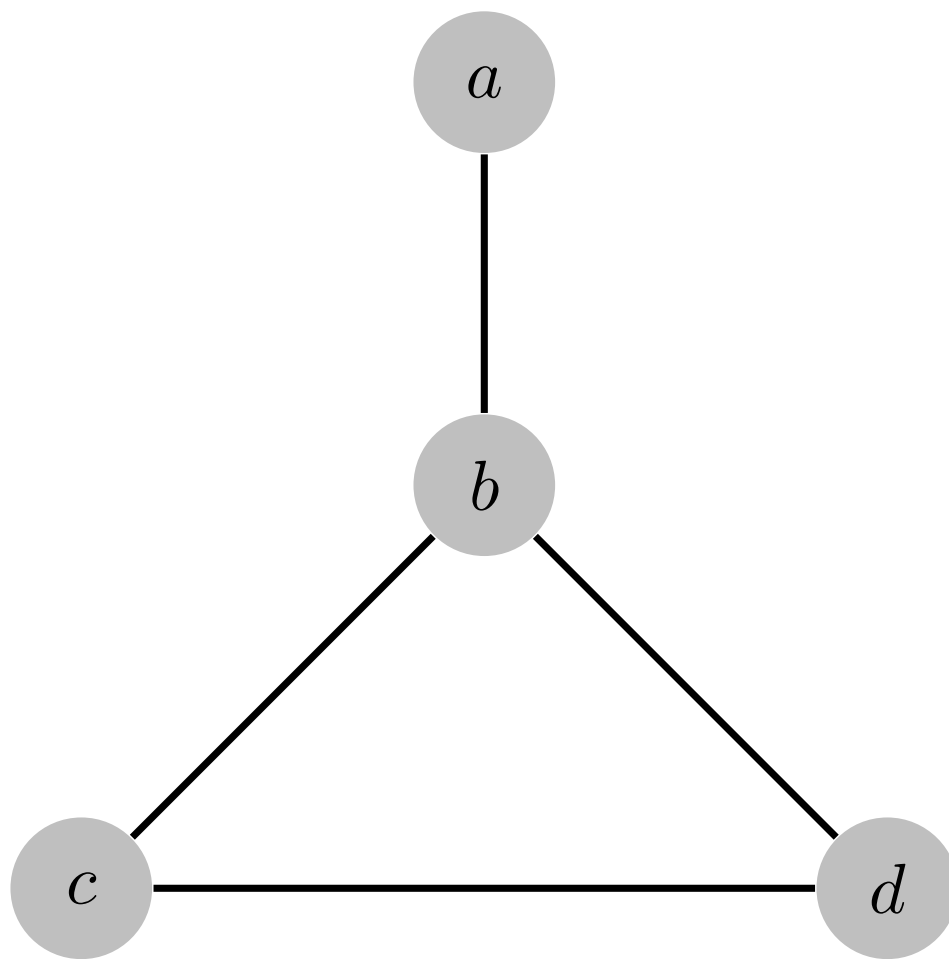
A graph  $G = (V, E)$  consists of:

- a set of **vertices** (nodes)  $V$
- a set of **edges** (links)  $E \subseteq V \times V$

This compact notation already encodes a modeling choice: what counts as a node, what counts as a relationship.

# Undirected Graphs

**Undirected Graph.** An undirected graph treats relationships as symmetric. An edge  $u, v$  connects two vertices without orientation:  $u, v = v, u$ . Source and target are not distinguished.

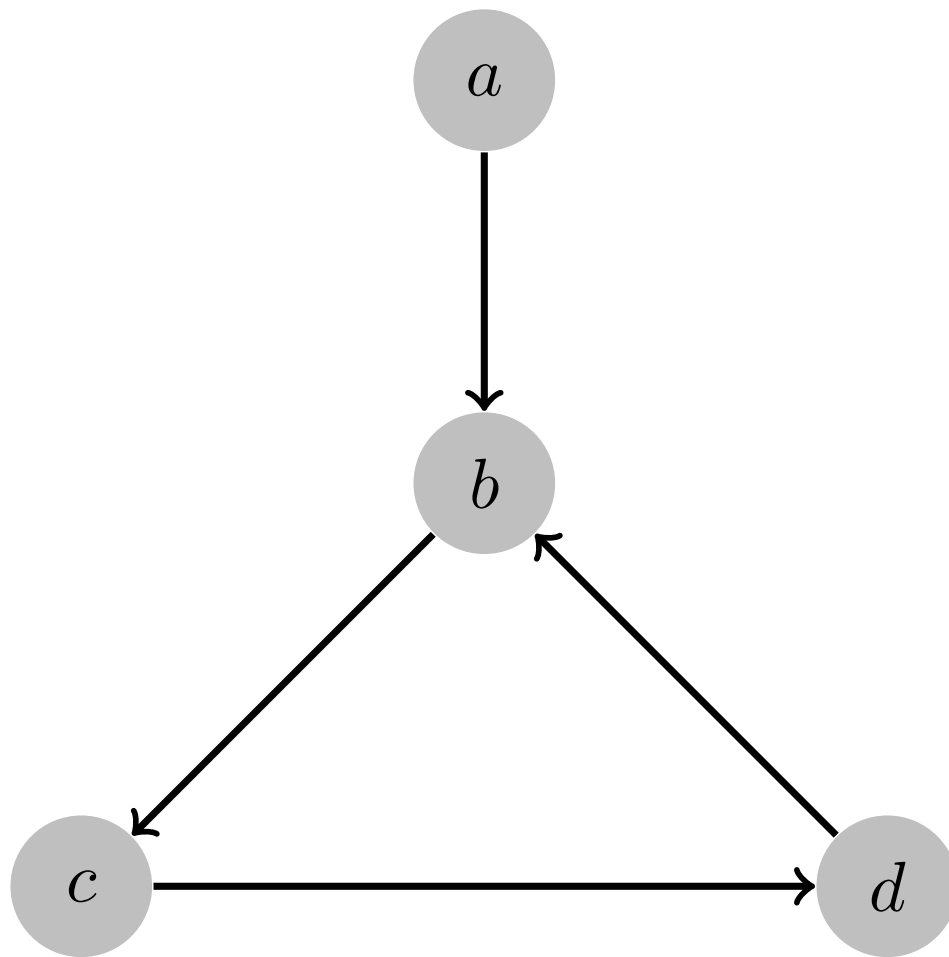


Edges have no arrows, meaning the relation is symmetric (e.g. friendship).



# Directed Graphs

**Directed Graph.** An edge  $(u, v) \in E$  has a strict orientation from  $u$  to  $v$ .  $(u, v)$  does not imply  $(v, u)$ . Directions matter whenever the relation is asymmetric (e.g. following, citing).



Edges have arrows indicating direction. The same set of nodes can yield very different models depending on this choice.

# Modeling Is a Choice

Before we analyse a graph, we decide how to turn a situation into one.

## From reality to graph model

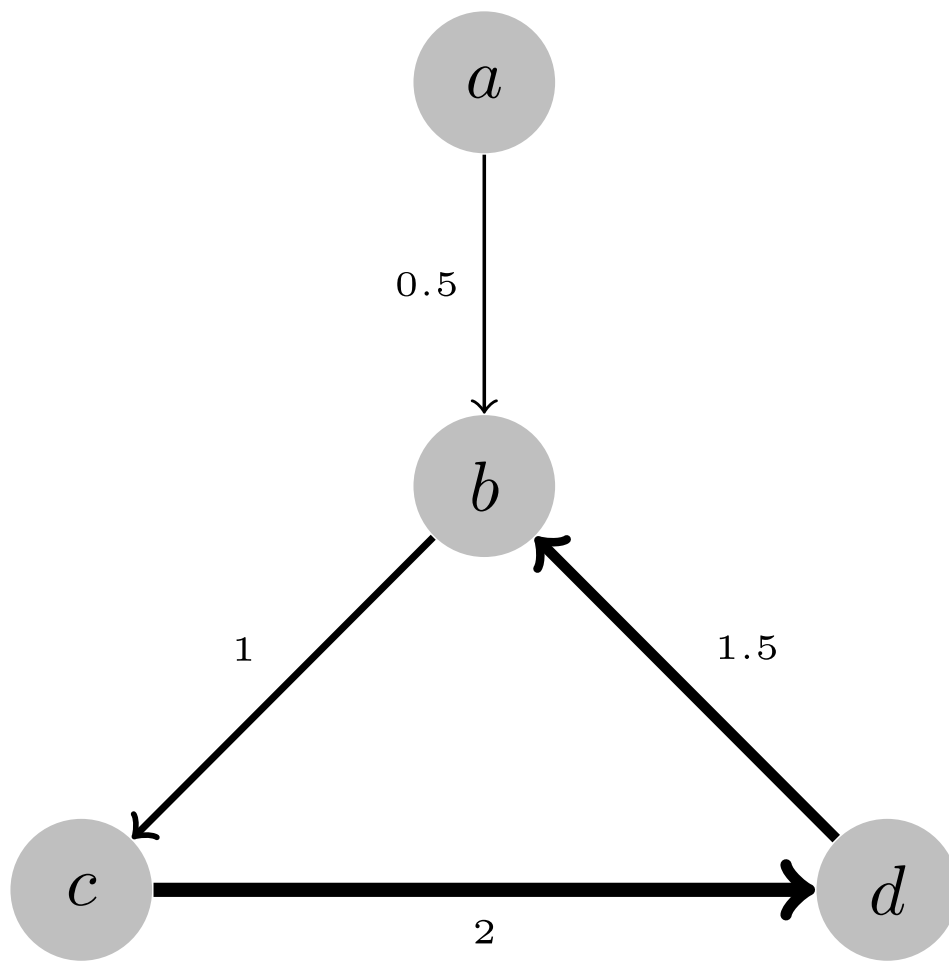


Example: an affiliation setting can become a bipartite graph, or a projection, depending on the question.

Graph theory gives us a **formal language**, but the first step is still **epistemic**: choose the entities, choose the relations, and decide what abstraction is good enough for the question.

# Weighted Graph

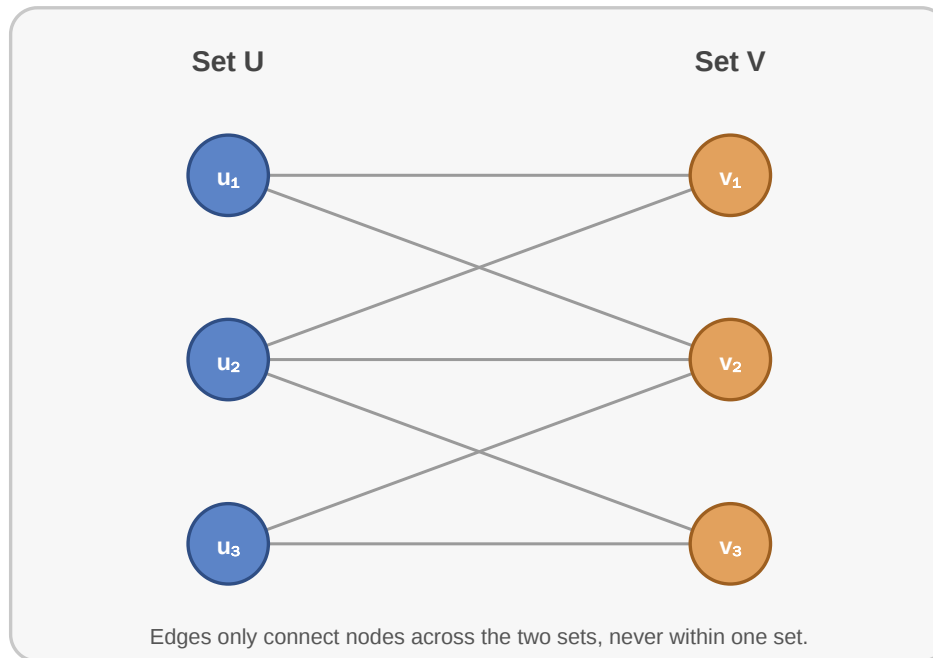
**Weighted Graph.** A graph equipped with a weight function  $w : E \rightarrow \mathbb{R}$ . The weight  $w(e)$  lets edges carry strength, cost, length, probability, or capacity.



Each edge carries a numerical value. The same topological structure can mean vastly different things depending on what the weights encode.

# Bipartite Graphs

**Bipartite Graph.** A graph  $G = (U, V, E)$  with two disjoint node sets ( $U \cap V = \emptyset$ ) where all edges connect a node from  $U$  to a node from  $V$ . Edges never connect two nodes within the same set.



Modelling of different kinds of nodes and limited edges. Useful when tracking relations between two inherently different sets of entities (e.g. students and courses, users and products).

# Graph Representations

How would you store (write down) a graph on paper?







# Graph Representations

How would you store (write down) a graph on paper?

## Graph representations

### Edge list

(u1, v1)  
(u1, v2)  
(u2, v2)  
(u3, v1)  
(u3, v2)

### Adjacency list

u1: v1, v2  
u2: v2  
u3: v1, v2  
v1: u1, u3  
v2: u1, u2, u3

### Adjacency matrix

|   |   |   |
|---|---|---|
| 0 | 1 | 1 |
| 0 | 0 | 1 |
| 1 | 1 | 1 |

Rows and columns encode the same graph in matrix form.

*Common representations are edge lists, adjacency lists, and adjacency matrices. They encode the same graph, but each supports different algorithms and scales differently.*

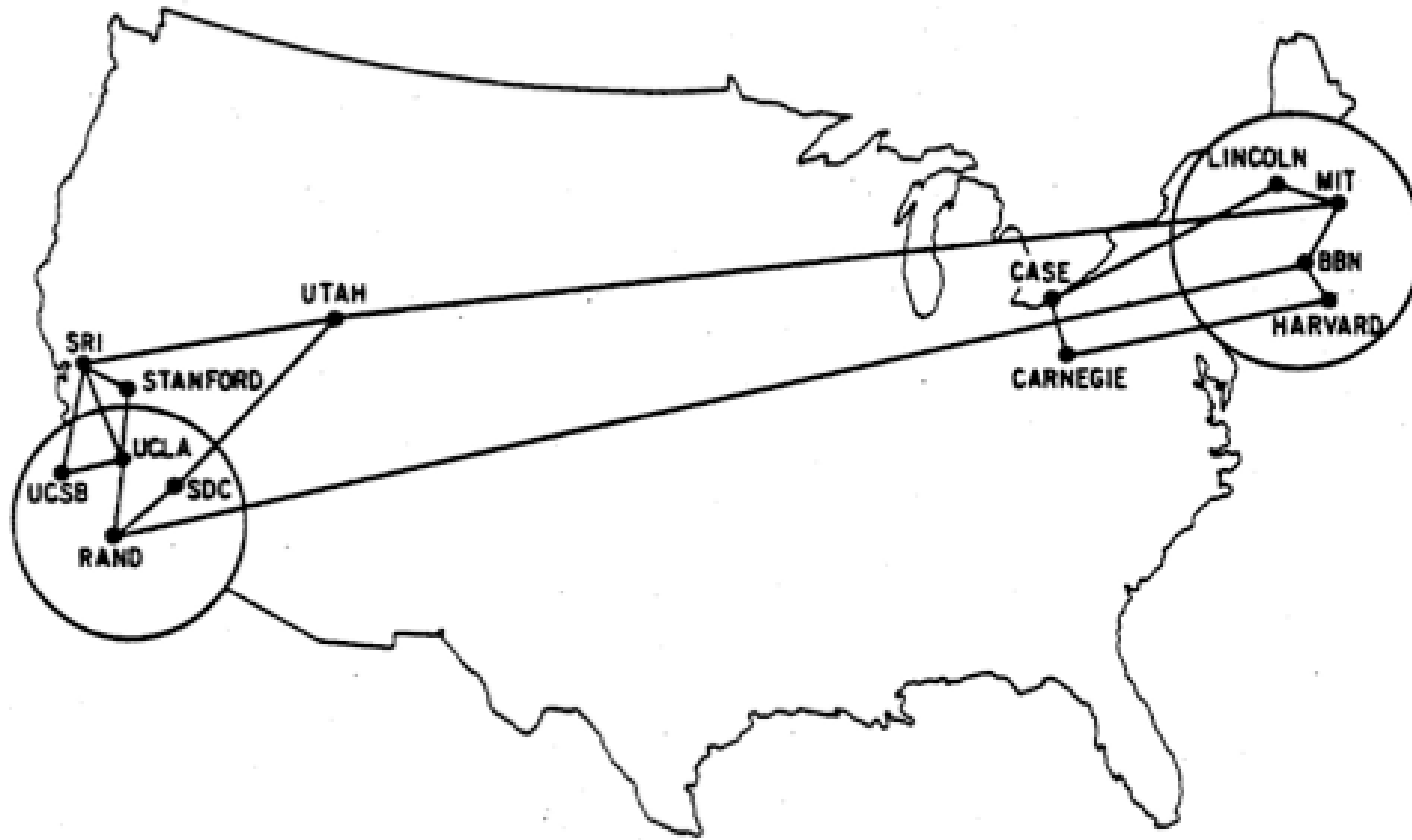
# Why Representations Matter

Different representations make different operations easier.

- edge lists are compact and easy to read
- adjacency lists are good for traversal
- adjacency matrices are good for algebraic methods

# Real-World Example: ARPANET

The ARPANET already shows how quickly a technical system becomes a graph with non-trivial structure (Easley & Kleinberg, 2010).



## Adjacency List (1970):

```
SRI:
    [UCSB, UCLA, Stanford, UTAH]
UCSB:
    [SRI, UCLA]
UCLA:
    [SRI, UCSB, RAND, Stanford]
RAND:
    [UCLA, BBN, SDC]
SDC:
    [RAND, UTAH]
Stanford: [SRI, UCLA]
Utah:
    [SRI, SDC, MIT]
Case:
    [Carnegie, Lincoln]
Carnegie: [Case, Harvard]
Harvard:
    [Carnegie, BBN]
BBN:
    [RAND, MIT, HARVARD]
MIT:
    [UTAH, BBN, Lincoln]
Lincoln:
    [Case, MIT]
```

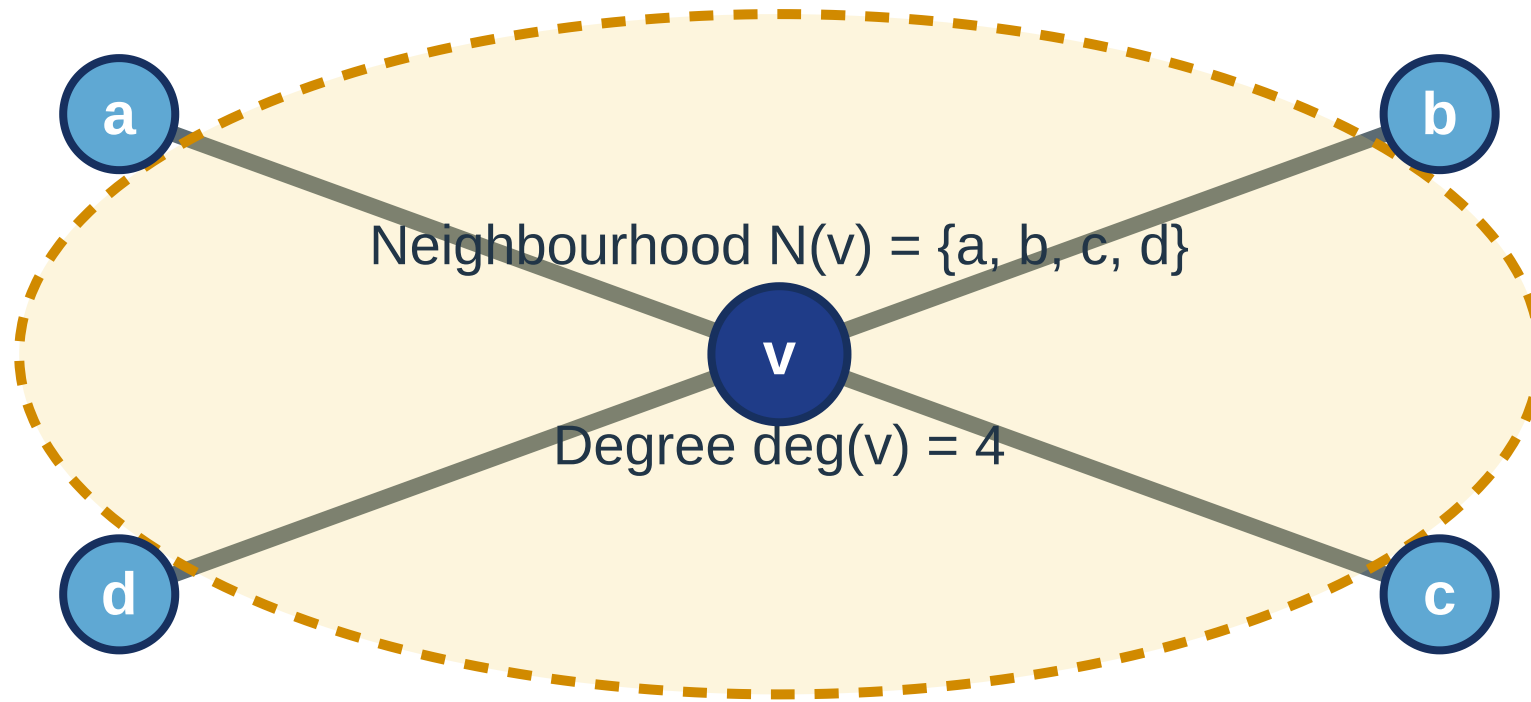
*ARPANET topology (1969–1977). Nodes are universities and research labs, edges are data links. One of the earliest computer networks.*

# Neighbourhood and Degree

**Neighbourhood.** The neighbourhood  $N(v)$  of a node  $v$  is the set of all nodes connected to  $v$  by an edge.

**Degree.** The degree  $\deg(v)$  is the number of edges incident to  $v$ . In simple graphs,  $\deg(v) = |N(v)|$ .





# Takeaway

Graph theory is useful because it gives a precise language for the properties we identified, but every graph still reflects a modeling choice that must be interpreted.



# Mini-Exercise 2.B.1

Consider a simple 5-node undirected graph: A-B, B-C, C-D, D-A, A-C. Here is an edge list and a function to convert it to an adjacency dictionary, but two key lines are missing. **What goes in the blanks?**

```
edges = [("A","B"), ("B","C"), ("C","D"), ("D","A"), ("A","C")]

def edges_to_adj(edges):
    adj = {}
    for u, v in edges:
        if u not in adj: adj[u] = []
        if v not in adj: adj[v] = []

        adj[u].append(____)
        adj[____].append(____)
    return adj
```

# Mini-Exercise 2.B.1 — Solution

```
edges = [("A","B"), ("B","C"), ("C","D"), ("D","A"), ("A","C")]

def edges_to_adj(edges):
    adj = {}
    for u, v in edges:
        if u not in adj: adj[u] = []
        if v not in adj: adj[v] = []

        adj[u].append(v)
        # Undirected graph implies symmetric edges
        adj[v].append(u)
    return adj
```

**Key insight:** Undirected graphs require symmetric entries. If a graph is directed, the second append line is omitted. Why should we not omit the second append line in undirected cases?

# Module 3

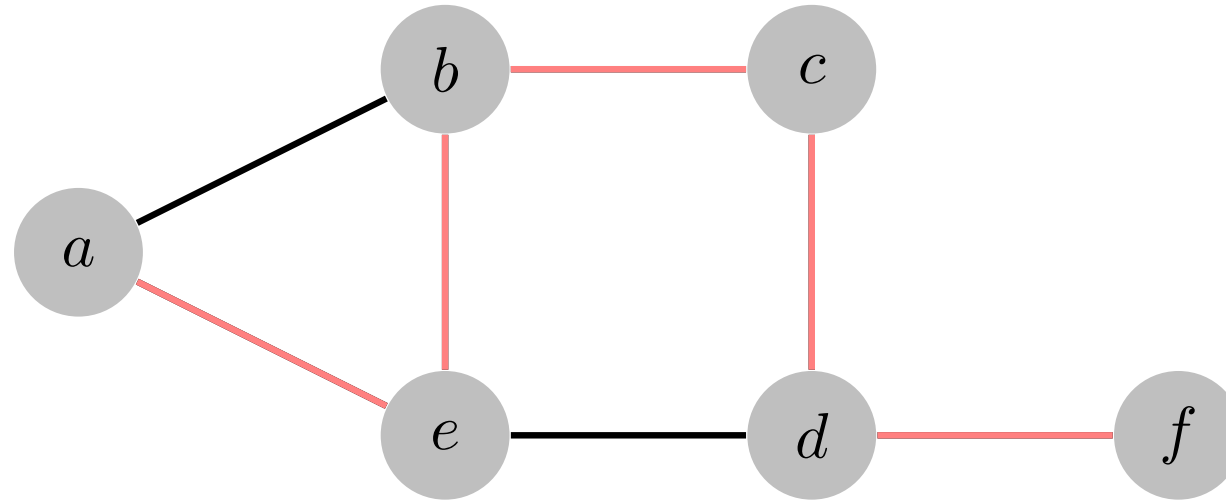
---

## Paths and Distance

**Guiding Question:** What does it mean for one node to "reach" another?

# Paths

A path is a sequence of vertices where each consecutive pair is connected by an edge.



*A path highlighted in a graph. The colored edges trace a route from a start node to an end node. The path length equals the number of edges traversed.*

# Walk, Path, Cycle

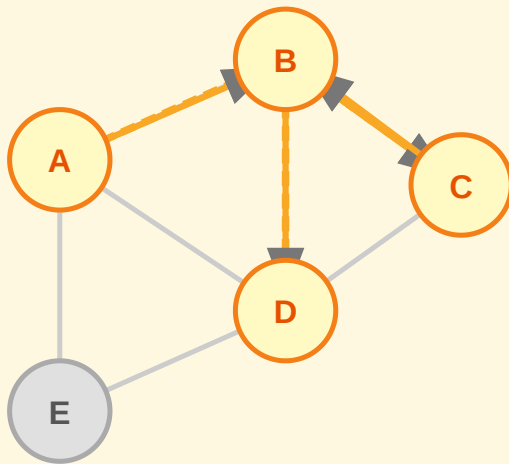
**Walk.** A walk of length  $k$  is a sequence  $v_0, v_1, \dots, v_k$  where  $(v_{i-1}, v_i) \in E$  for all  $i$ . Vertices may be repeated.

**Path.** A path is a walk where all vertices are distinct:  $v_i \neq v_j$  for  $i \neq j$ . Also called *simple path*.

**Cycle.** A cycle is a walk  $v_0, v_1, \dots, v_k$  with  $v_0 = v_k$  and all intermediate vertices distinct.

### Walk

May revisit vertices or edges

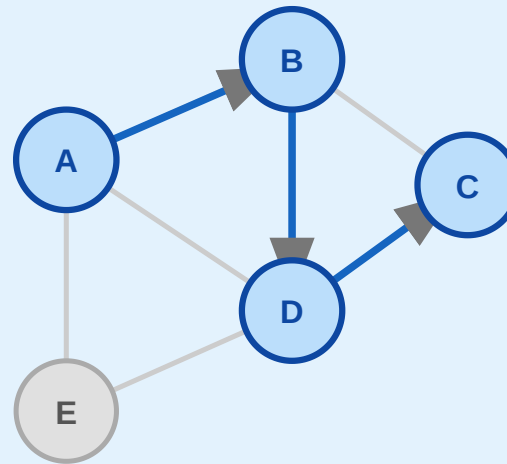


Sequence:  $A \rightarrow B \rightarrow C \rightarrow B \rightarrow D$

B is visited twice — valid walk!

### Path

All vertices distinct (simple path)

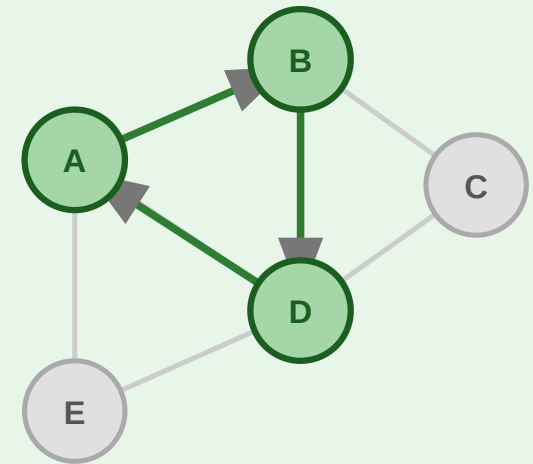


Sequence:  $A \rightarrow B \rightarrow D \rightarrow C$

Each vertex appears exactly once

### Cycle

Closed walk, start = end, no other repeats



Sequence:  $A \rightarrow B \rightarrow D \rightarrow A$

Returns to start, no repeated intermediate vertex

**Directed vs. undirected.** In an undirected graph, an edge can be traversed in either direction. In a directed graph, each step must follow the edge direction.

# Path Length, Shortest Path, Diameter

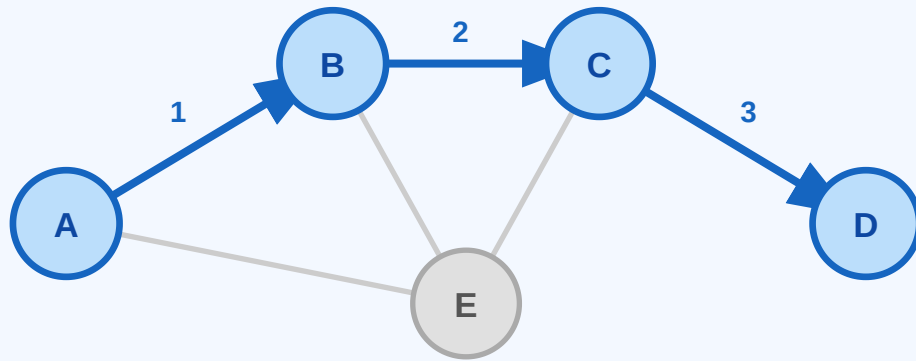
**Path length.** The length of a path  $v_0, \dots, v_k$  is  $k$  (number of edges). In a weighted graph:  $\sum_{i=1}^k w(v_{i-1}, v_i)$ .

**Shortest path.**  $\text{dist}(u, v) = \min\{k \mid \text{path of length } k \text{ from } u \text{ to } v\}$ . If no path exists,  $\text{dist}(u, v) = \infty$ .

**Diameter.**  $\text{diam}(G) = \max_{u, v \in V} \text{dist}(u, v)$  — the longest shortest path in the graph.

## Path Length

Number of edges in the path

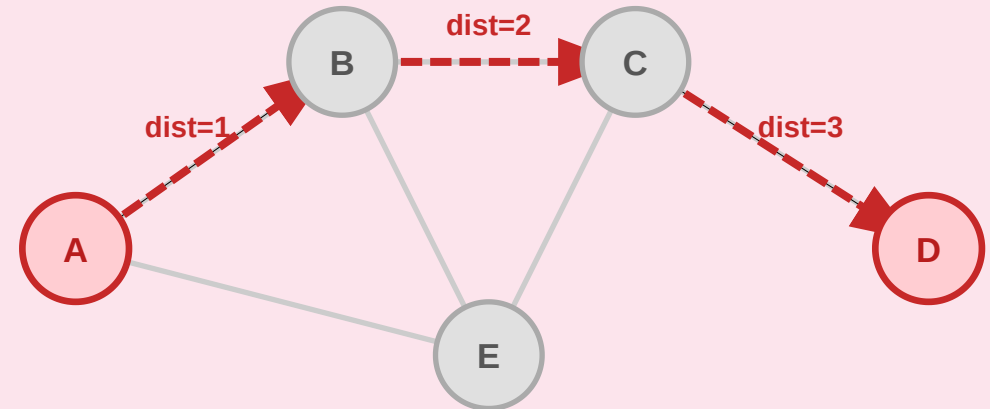


Path  $A \rightarrow B \rightarrow C \rightarrow D$  has length 3  
(3 edges traversed)

*Weighted: length = sum of edge weights*

## Diameter

Longest shortest path in the graph



$\text{diam}(G) = \text{dist}(A, D) = 3$

The farthest pair determines the diameter

*In directed graphs,  $\text{dist}(u, v) \neq \text{dist}(v, u)$  in general*



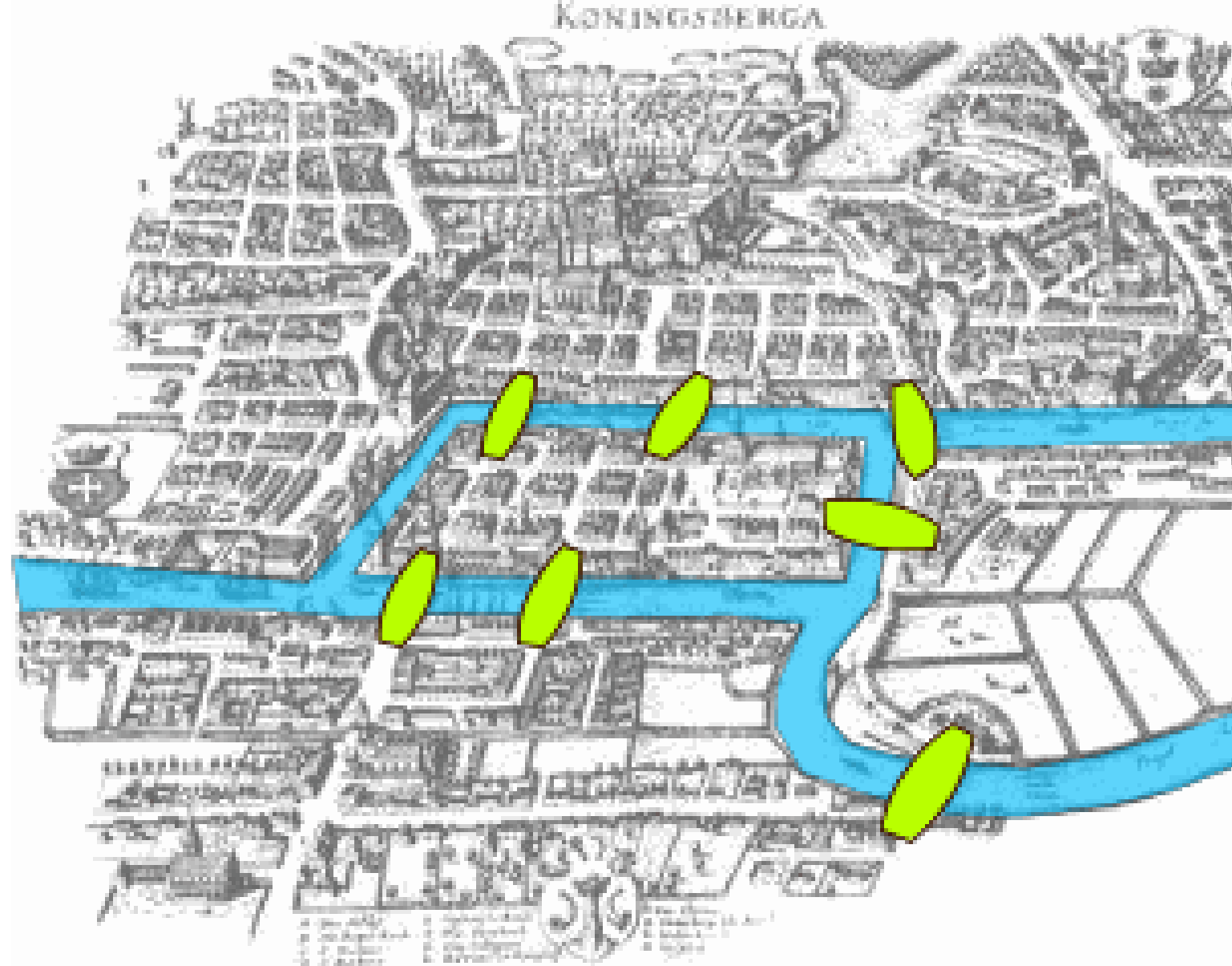
# Why Paths Matter

- they formalize reachability
- they give us a notion of distance
- they underpin routing, diffusion, and traversal algorithms

# A Classical Motivation

The Königsberg bridges problem is one of the classic entry points into graph theory.





# Eulerian Path and Circuit

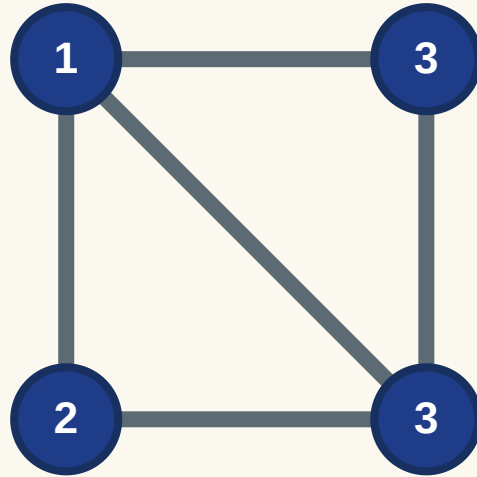
**Eulerian Path.** A walk that traverses every edge of a graph exactly once. It exists if and only if exactly zero or two vertices have an odd degree.

**Eulerian Circuit.** An Eulerian path that starts and ends on the same vertex. It exists if and only if every vertex has an even degree.



## Eulerian Path

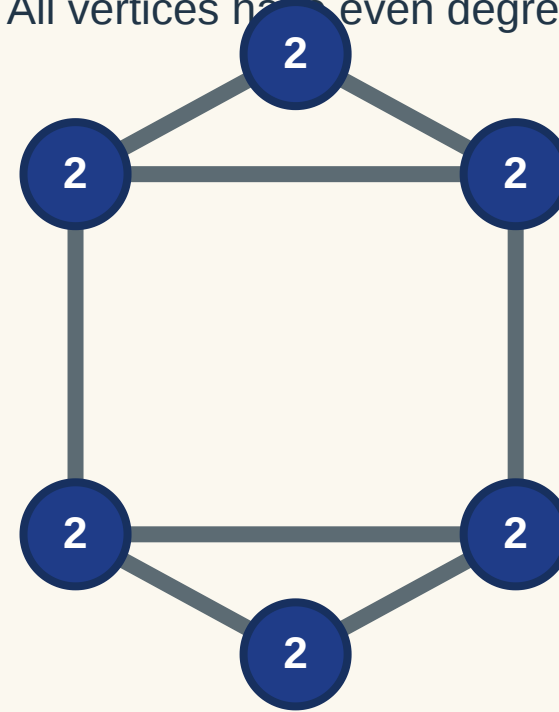
Exactly two odd-degree vertices



Odd degrees: 2 endpoints

## Eulerian Circuit

All vertices have even degree



Closed walk possible

# Takeaway

Paths turn connectivity into something analyzable: reachability, distance, redundancy, and traversal constraints.

# Mini-Exercise 2.C.1

Consider a 5-node graph:  $\text{adj} = \{1: [2, 3], 2: [1, 4], 3: [1, 4], 4: [2, 3, 5], 5: [4]\}$

Here is a recursive function to find *any* path between two nodes, but the crucial recursive step is missing. **What goes in the blank?**

```
def find_path(graph, start, end, path=[]):  
    path = path + [start]  
    if start == end: return path  
    for node in graph.get(start, []):  
        if node not in path:  
            # How do we continue the search?  
            result = -----  
            if result is not None: return result  
    return None
```

# Mini-Exercise 2.C.1 — Solution

**Key insight:** DFS explores deep first and finds *any* path. How would we change the approach to guarantee finding the path?



# Mini-Exercise 2.C.1 — Solution

```
def find_path(graph, start, end, path=[]):
    path = path + [start]
    if start == end: return path
    for node in graph.get(start, []):
        if node not in path:
            # Recursive call on the neighbor
            result = find_path(graph, node, end, path)
            if result is not None: return result
    return None
```

**Key insight:** DFS explores deep first and finds *any* path. How would we change the approach to guarantee finding the path?

# Mini-Exercise 2.C.1 — Solution

```
def find_path(graph, start, end, path=[]):
    path = path + [start]
    if start == end: return path
    for node in graph.get(start, []):
        if node not in path:
            # Recursive call on the neighbor
            result = find_path(graph, node, end, path)
            if result is not None: return result
    return None
```

**Key insight:** DFS explores deep first and finds *any* path. How would we change the approach to guarantee finding the *shortest* path?

# Module 4

---

## Breadth-First Search

**Guiding Question:** How can we systematically find a node — or the shortest route to it — in a large graph?

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

1. Initialize a  $Q$  and add  $s$
2. Mark  $s$  as
  - Uses a (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

1. Initialize a  $Q$  and add  $s$
2. Mark  $s$  as

- Uses a (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a  $Q$  and add  $s$
2. Mark  $s$  as visited
  - Uses a (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

*(Input: Graph  $G = (V, E)$ , source node  $s$ )*

1. Initialize a **queue**  $Q$  and add  $s$
2. Mark  $s$  as
  - Uses a (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

*(Input: Graph  $G = (V, E)$ , source node  $s$ )*

1. Initialize a **queue**  $Q$  and add  $s$
  2. Mark  $s$  as **visited**
- Uses a (First-In, First-Out) queue.



# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a **queue**  $Q$  and add  $s$
2. Mark  $s$  as **visited**
3. While  $Q$  is not empty:
  - For each neighbor  $v$  of  $u$ : if  $v$  is **not visited**, mark  $v$  as visited and add  $v$  to the back of  $Q$
  - Uses a **queue** (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a **queue**  $Q$  and add  $s$
  2. Mark  $s$  as **visited**
  3. While  $Q$  is not empty:
    - Extract current node  $u$  from the front of  $Q$
    - For each neighbor  $v$  of  $u$ : if  $v$  is not visited, mark  $v$  as visited and add  $v$  to the back of  $Q$
- Uses a queue (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a **queue**  $Q$  and add  $s$
  2. Mark  $s$  as **visited**
  3. While  $Q$  is not empty:
    - Extract current node  $u$  from the front of  $Q$
    - For each neighbor  $v$  of  $u$ : if  $v$  is **not visited**, mark  $v$  as visited and add  $v$  to the back of  $Q$
- Uses a (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a **queue**  $Q$  and add  $s$
2. Mark  $s$  as **visited**
3. While  $Q$  is not empty:
  - Extract current node  $u$  from the front of  $Q$
  - For each neighbor  $v$  of  $u$ : if  $v$  is **not visited**, mark  $v$  as visited and add  $v$  to the back of  $Q$

## Key Properties:

- Uses a (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a **queue**  $Q$  and add  $s$
2. Mark  $s$  as **visited**
3. While  $Q$  is not empty:
  - Extract current node  $u$  from the front of  $Q$
  - For each neighbor  $v$  of  $u$ : if  $v$  is **not visited**, mark  $v$  as visited and add  $v$  to the back of  $Q$

## Key Properties:

- Uses a **FIFO** (First-In, First-Out) queue.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a **queue**  $Q$  and add  $s$
2. Mark  $s$  as **visited**
3. While  $Q$  is not empty:
  - Extract current node  $u$  from the front of  $Q$
  - For each neighbor  $v$  of  $u$ : if  $v$  is **not visited**, mark  $v$  as visited and add  $v$  to the back of  $Q$

## Key Properties:

- Uses a **FIFO** (First-In, First-Out) queue.
- Discovers nodes strictly layer by layer.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

1. Initialize a **queue**  $Q$  and add  $s$
2. Mark  $s$  as **visited**
3. While  $Q$  is not empty:
  - Extract current node  $u$  from the front of  $Q$
  - For each neighbor  $v$  of  $u$ : if  $v$  is **not visited**, mark  $v$  as visited and add  $v$  to the back of  $Q$

## Key Properties:

- Uses a **FIFO** (First-In, First-Out) queue.
- Discovers nodes strictly layer by layer.
- Runs in  $O(|V| + |E|)$  time.

# Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

## Algorithm: BFS

(Input: Graph  $G = (V, E)$ , source node  $s$ )

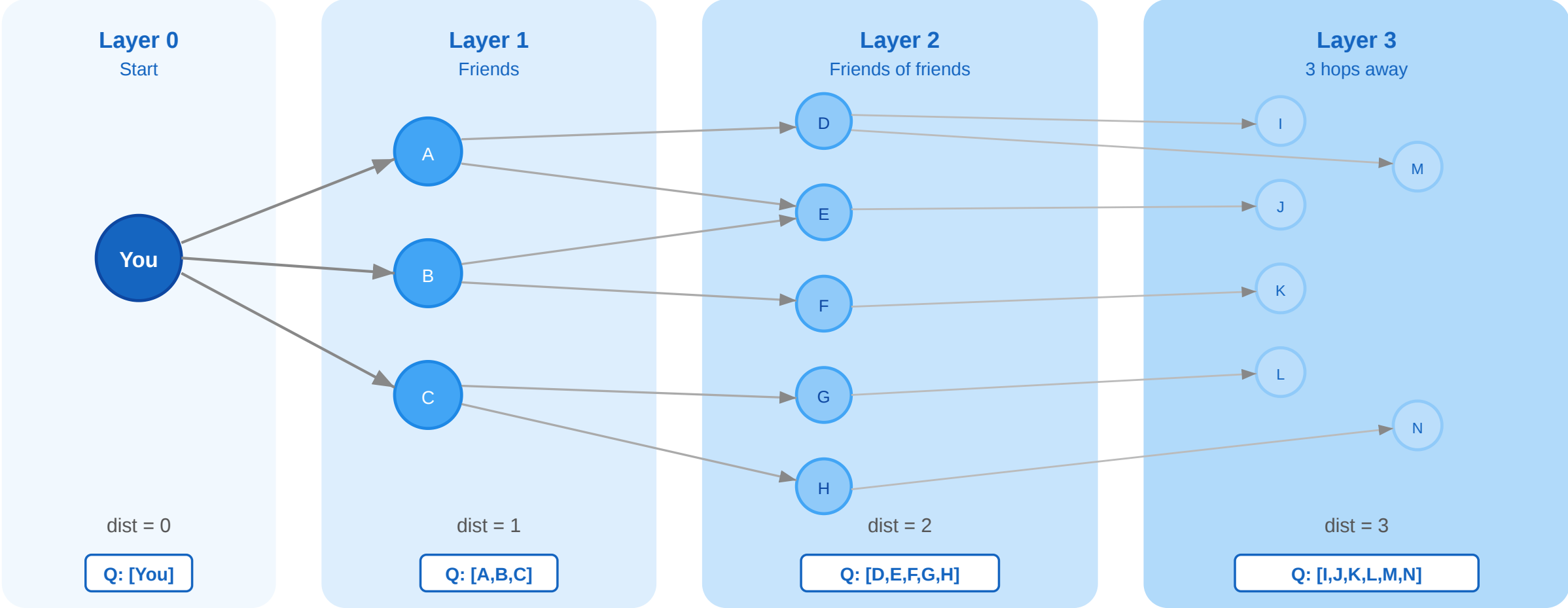
1. Initialize a **queue**  $Q$  and add  $s$
2. Mark  $s$  as **visited**
3. While  $Q$  is not empty:
  - Extract current node  $u$  from the front of  $Q$
  - For each neighbor  $v$  of  $u$ : if  $v$  is **not visited**, mark  $v$  as visited and add  $v$  to the back of  $Q$

## Key Properties:

- Uses a **FIFO** (First-In, First-Out) queue.
- Discovers nodes strictly layer by layer.
- Runs in  $O(|V| + |E|)$  time.
- Useful for shortest paths, components, and reachability.

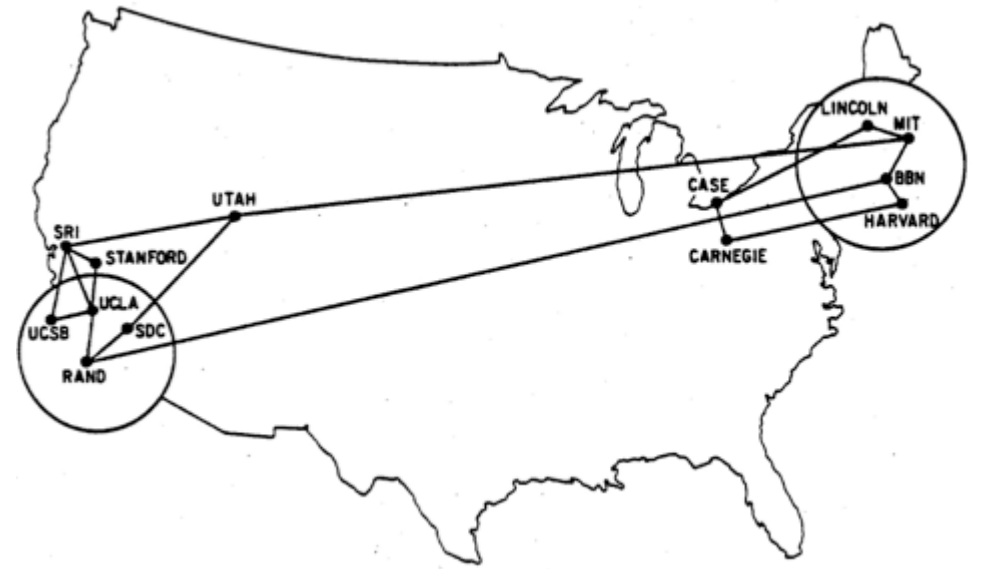
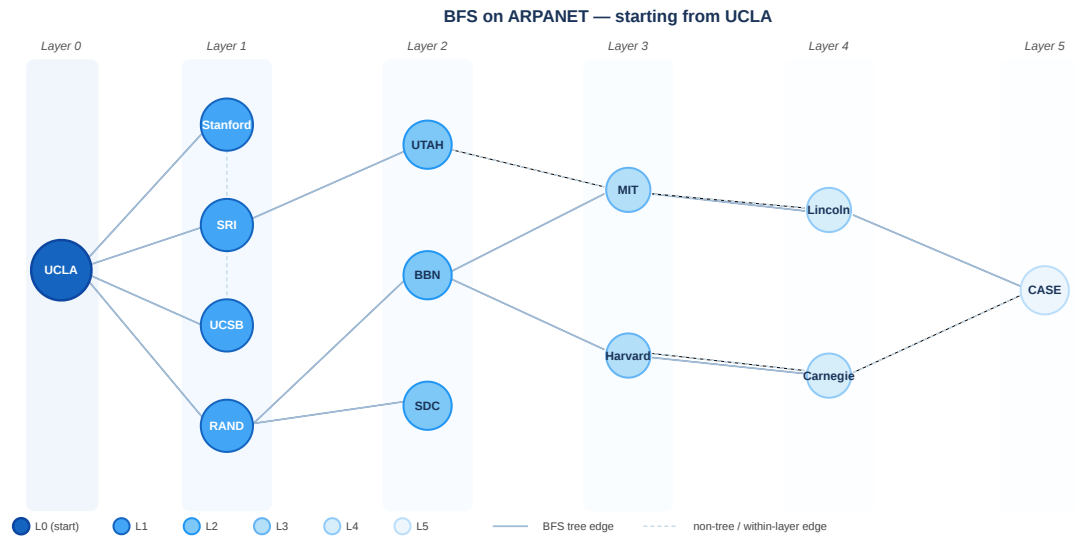


# Visual Intuition



***BFS layer-by-layer expansion.** Starting from "You" (layer 0), BFS discovers all direct neighbors (layer 1), then all not-yet-visited neighbors of those (layer 2), and so on.*

# BFS on the ARPANET

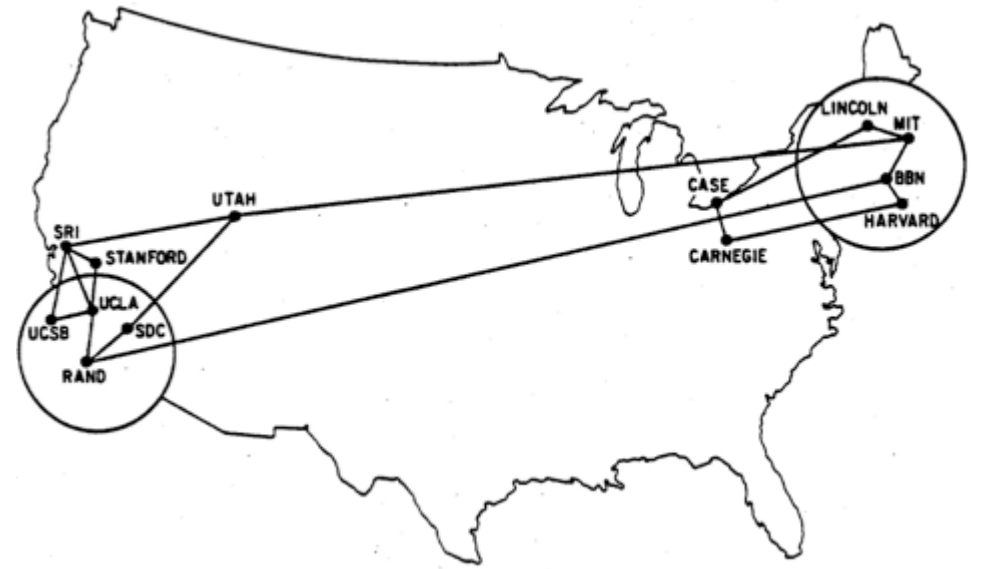
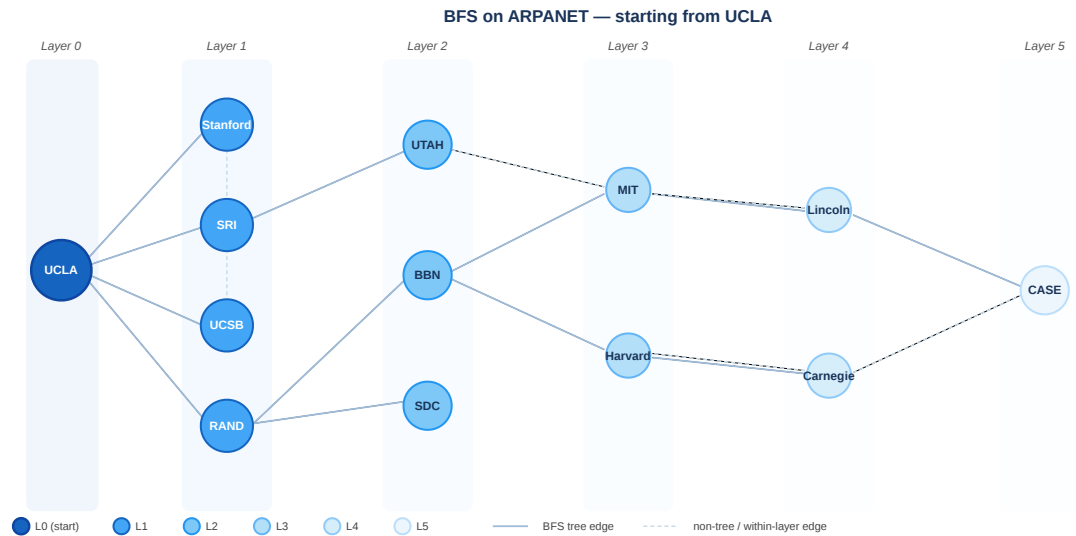


**Left:** BFS applied to the ARPANET topology starting from UCLA. Node color encodes BFS layer (distance). **Right:** The physical ARPANET map.

**Discussion:** The BFS visualization shows some nodes with multiple incoming edges from the previous layer. Is this possible in a strict BFS spanning tree?

*Answer: No. A true BFS tree has exactly one parent per node. Drawing shortest-path edges creates a "level graph", not a spanning tree!*

# BFS on the ARPANET



**Left:** BFS applied to the ARPANET topology starting from UCLA. Node color encodes BFS layer (distance). **Right:** The physical ARPANET map.

**Discussion:** The BFS visualization shows some nodes with multiple incoming edges from the previous layer. Is this possible in a strict BFS spanning tree?

*Answer: No. A true BFS tree has exactly one parent per node. Drawing all shortest-path edges creates a "level graph", not a spanning tree!*

# Complexity

- Queue-based BFS runs in  $O(|V| + |E|)$  — each vertex and each edge is visited at most once
- This makes BFS the standard exact method for shortest paths in **unweighted** graphs

**Directed graphs:** BFS works identically — it follows edges in their given direction.

**Weighted graphs:** BFS does **not** give shortest paths when edges have different weights. For weighted shortest paths, use Dijkstra's algorithm.

# Takeaway

BFS matters because a simple local rule produces exact shortest-path information in unweighted graphs.



# Dijkstra's Algorithm

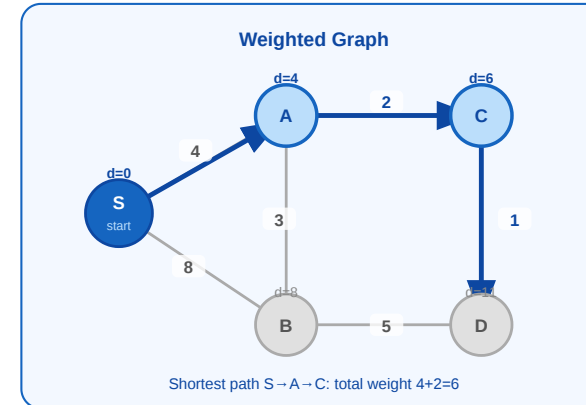
BFS explores layer by layer. For **weighted** graphs, we use Dijkstra's algorithm.

## Algorithm: Dijkstra

(Input: Graph  $G = (V, E)$ , source  $s$ , weights  $w \geq 0$ )

1. Set distance to  $s$  as 0 and all others as  $\infty$ .
2. Initialize a **priority queue**  $PQ$  with all nodes.
3. While  $PQ$  is not empty:
  - Extract node  $u$  with the minimum distance.
  - For each neighbor  $v$  of  $u$ :
    - If path through  $u$  is shorter: update  $v$ 's distance.

Dijkstra's Algorithm — Weighted Shortest Path Example



### Algorithm Steps

Idea: greedily settle shortest distances

1. Initialize:  $\text{dist}[S]=0$ ,  $\text{dist}[\text{all others}]=\infty$
2. Use a priority queue (min-heap) ordered by dist
3. Extract the node  $u$  with smallest dist
4. For each neighbor  $v$  of  $u$ :  
if  $\text{dist}[u] + w(u,v) < \text{dist}[v]$ : update  $\text{dist}[v]$
5. Repeat until all nodes settled

Complexity:  $O((|V| + |E|) \log |V|)$   
with a binary min-heap priority queue

⚠ Requires non-negative edge weights  
Use Bellman-Ford for negative weights

## BFS vs. Dijkstra.

*BFS explores identical layers. Dijkstra uses a priority queue to always expand the currently cheapest unvisited node. Complexity:*

$$O((|V| + |E|) \log |V|)$$

## Mini-Exercise 2.D.1

Consider our 5-node graph: `adj = {1: [2, 3], 2: [1, 4], 3: [1, 4], 4: [2, 3, 5], 5: [4]}`

Here is a BFS function to calculate the shortest path *distance* from `start` to all reachable nodes. **What goes in the two blanks?**

```
from collections import deque

def bfs(graph, start):
    distances = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbor in graph.get(current, []):
            if neighbor not in distances:
                distances[neighbor] = _____
                queue._____(neighbor)
    return distances
```

## Mini-Exercise 2.D.1 — Solution

Because the queue is FIFO, we always finish layer  $k$  before starting layer  $k + 1$ . The first time we see a node, we have found its shortest path.

# Mini-Exercise 2.D.1 — Solution

```
from collections import deque

def bfs(graph, start):
    distances = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbor in graph.get(current, []):
            if neighbor not in distances:
                distances[neighbor] = distances[current] + 1
                queue.append(neighbor)
    return distances
```

Because the queue is FIFO, we always finish layer  $k$  before starting layer  $k + 1$ . The first time we see a node, we have found its shortest path.

# Mini-Exercise 2.D.1 — Solution

```
from collections import deque

def bfs(graph, start):
    distances = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbor in graph.get(current, []):
            if neighbor not in distances:
                distances[neighbor] = distances[current] + 1
                queue.append(neighbor)
    return distances
```

**Key insight:** Because the queue is FIFO, we always finish layer  $k$  before starting layer  $k + 1$ . The first time we see a node, we have found its shortest path.

# Module 5

---

## Connectivity

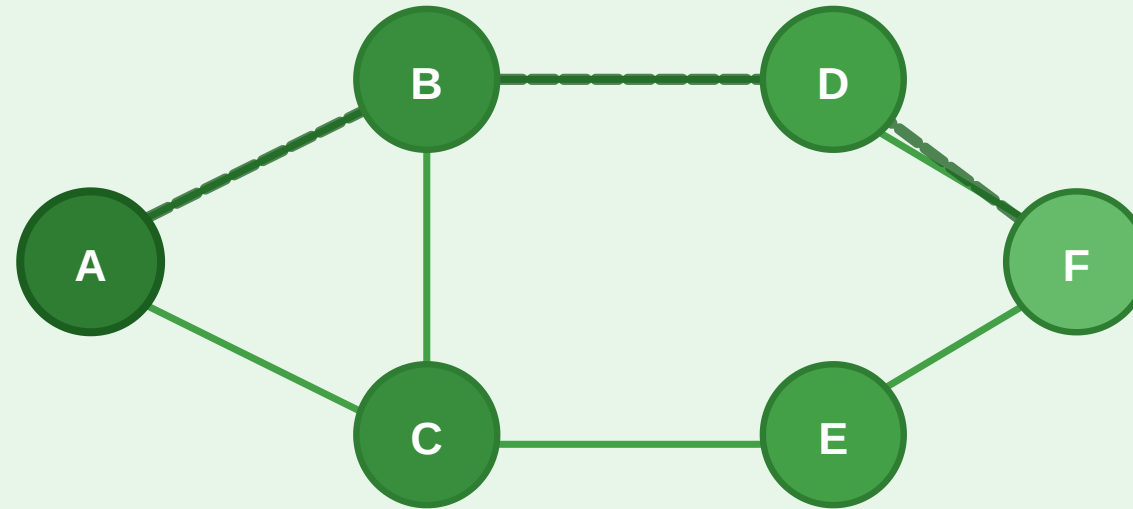
**Guiding Question:** When is a network still one system — and when has it split apart?

# Connectedness

**Connected (undirected).** Let  $G = (V, E)$  be a graph.  $G$  is *connected* if for every pair  $u, v \in V$  there exists a path from  $u$  to  $v$ .

## Connected

A path exists between every pair of nodes

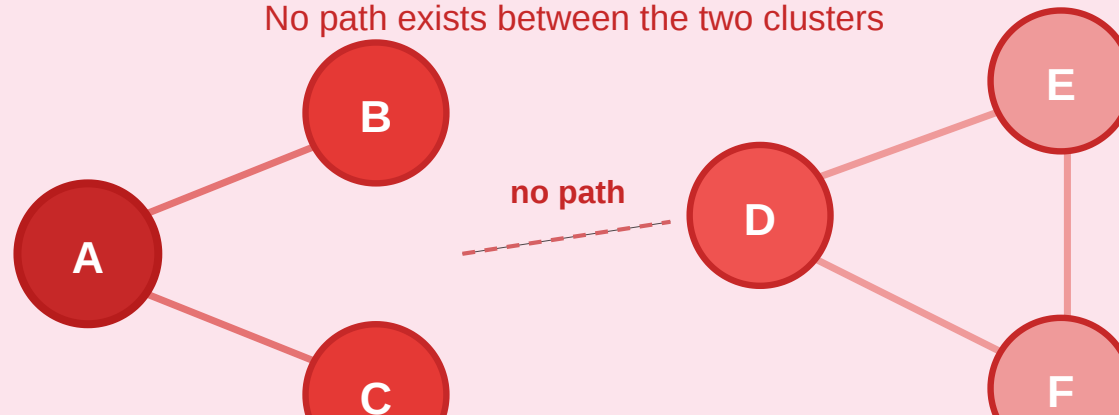


Path  $A \rightarrow B \rightarrow D \rightarrow F$  shown — one connected component

vs.

## Disconnected

No path exists between the two clusters





# Connected Components

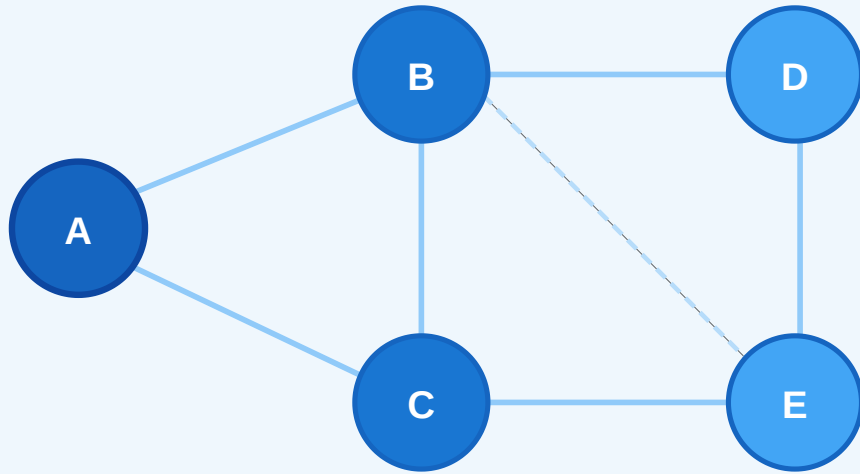
Connected components are maximal connected subgraphs.

**Connected component.** A *connected component* of  $G$  is a subgraph  $H \subseteq G$  such that:

1.  $H$  is **connected** (every pair of vertices in  $H$  has a path within  $H$ ), and
2.  $H$  is **maximal** — no vertex outside  $H$  has a path to any vertex in  $H$ .

### Component 1

Every node reachable from every other node



5 nodes, fully internally connected  
No edge reaches Component 2

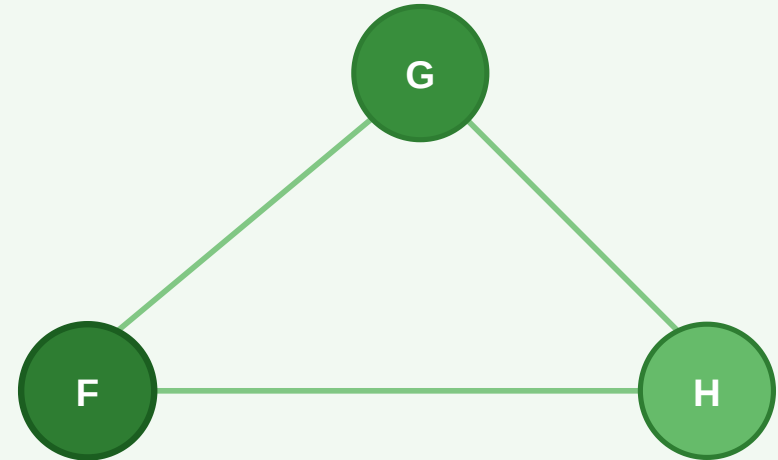
no edges



between

### Component 2

Every node reachable from every other node



3 nodes, fully internally connected  
No edge reaches Component 1

*Each colored cluster is a maximal connected subgraph — meaning it cannot be extended without losing separation.*

# Connectedness in Directed Graphs

How does connectivity change when edges have direction?

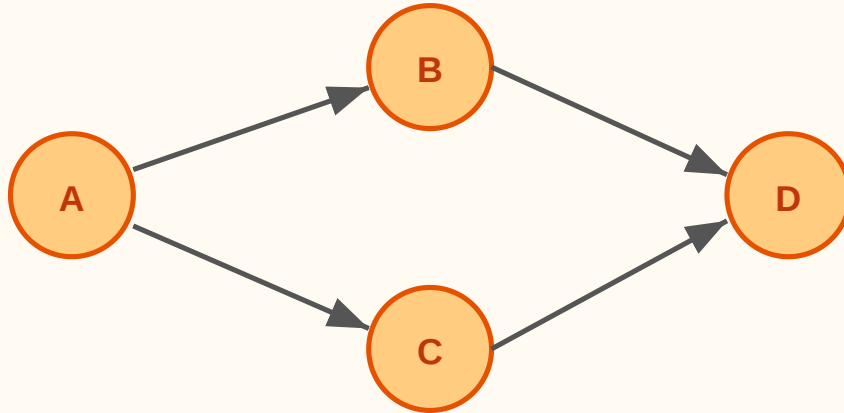
**Strongly connected.** For every pair  $u, v \in V$  there is a directed path from  $u$  to  $v$  **and** from  $v$  to  $u$ .

**Weakly connected.** The graph is connected when all directed edges are replaced with undirected edges.



### Weakly Connected

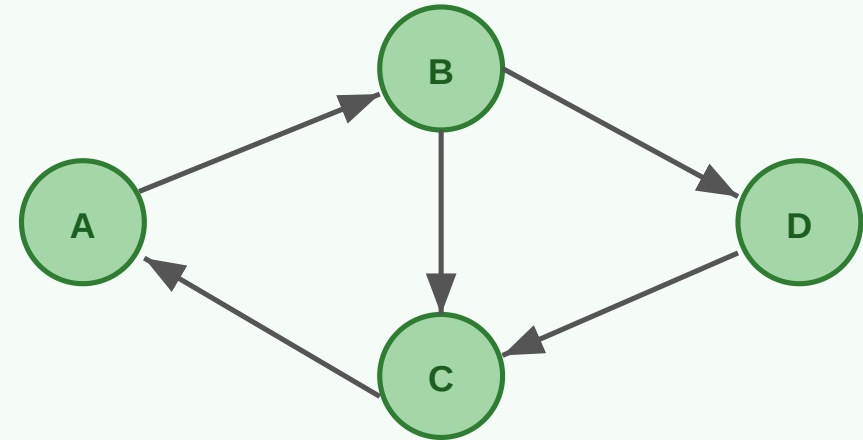
(directed, NOT strongly connected)



A can reach D, but D cannot reach A

### Strongly Connected

(directed, every pair mutually reachable)

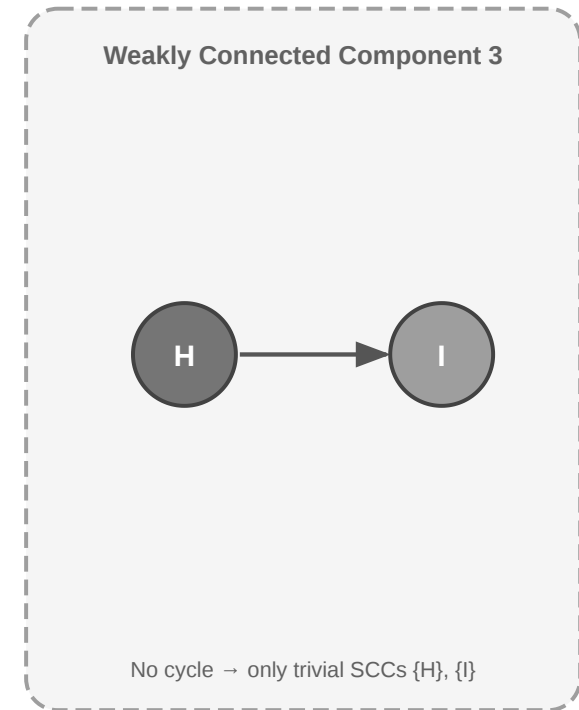
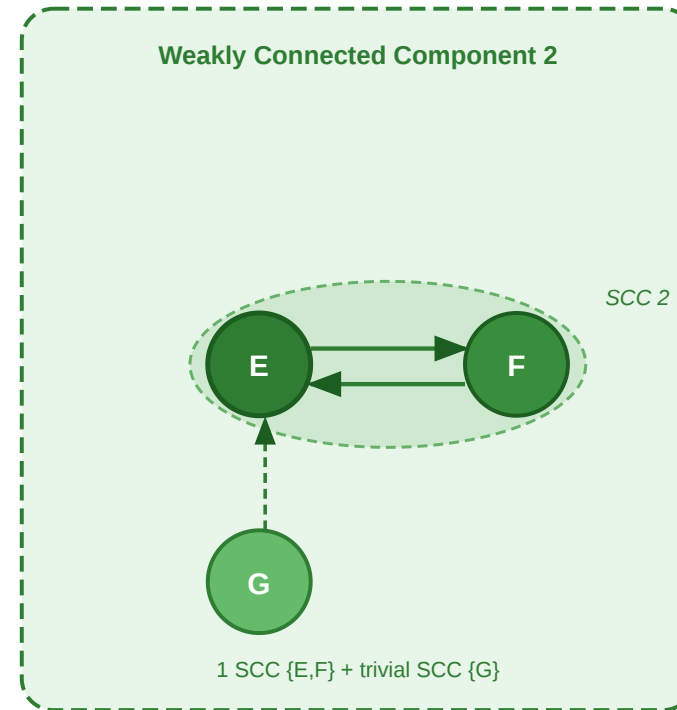
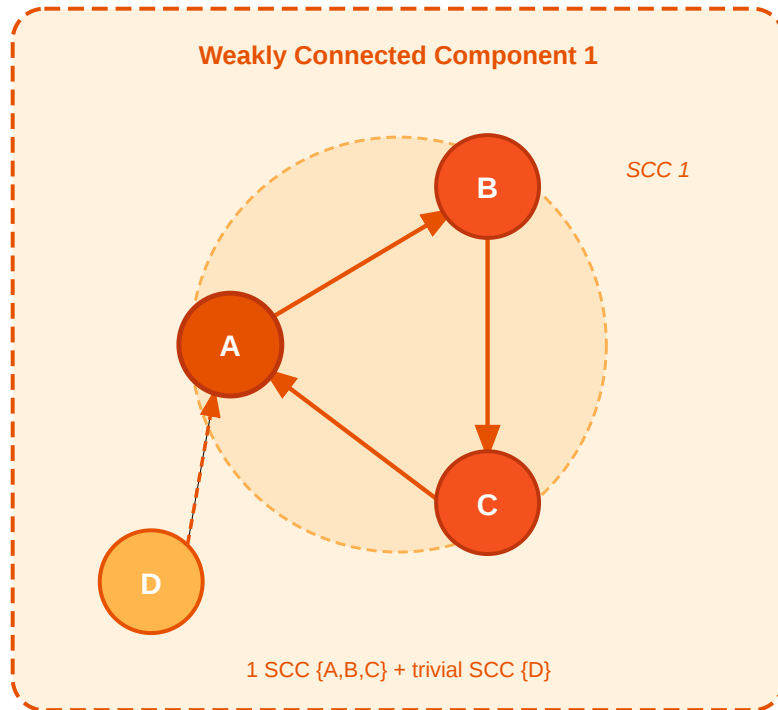


Every node can reach every other node

**Left:** Weakly connected — *D* cannot reach *A* following arrows. **Right:** Strongly connected — every node reaches every other.

# Directed Graph Components

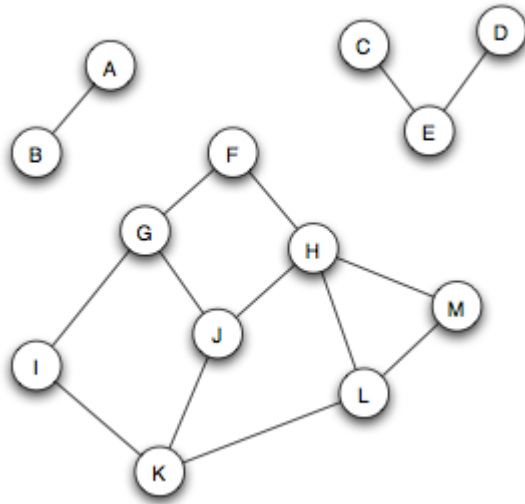
A directed graph has **weakly** and **strongly** connected components — reusing the definitions we already know.



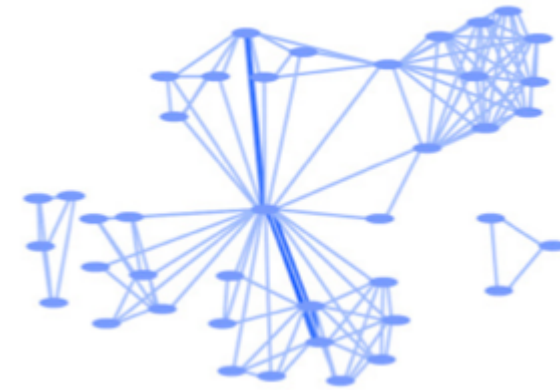
*3 weakly connected components (dashed borders). 2 strongly connected components (shaded halos). Nodes D, G, H, I each form their own trivial SCC.*

# Real-World Connectivity

Many real networks contain one large giant component and several much smaller disconnected parts (Easley & Kleinberg, 2010).



Source: Easley & Kleinberg 2010

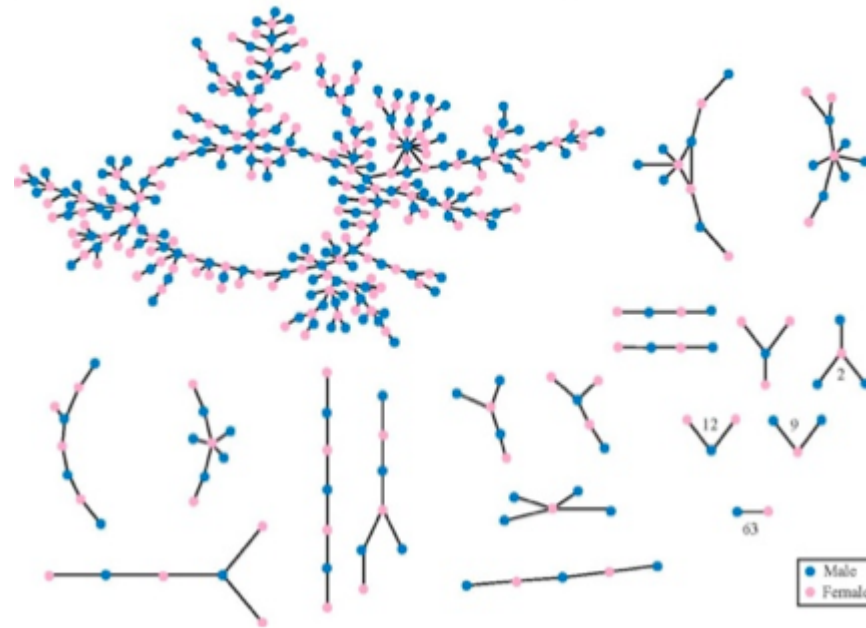


Source: Easley & Kleinberg 2010

**Left:** A network with three distinct connected components. **Right:** A collaboration network where one giant component dominates while smaller isolated groups exist at the periphery.

# Giant Components

**Giant component.** A connected component that is significantly larger than all others, usually containing a macroscopic fraction of all vertices. Appears quite often in real world networks.



Source: Easley & Kleinberg 2010



*The large central structure is the giant component. The small isolated clusters at the edges represent disconnected social groups.*

# Takeaway

Connected components and giant components tell us whether interaction, diffusion, or coordination can happen across the whole graph or only inside local regions.

# Mini-Exercise 2.E.1

Build on your BFS implementation to find connected components:

1. What is the strategy for using BFS to find connected components?
2. Write a function `find_components(graph)` that returns a list of sets, where each set is a connected component
3. Test it on a graph with **two disconnected parts**
4. **Discussion Points**
  - How does it change for directed graphs?
  - Can removing a node from a connected component make it disconnected?

# Mini-Exercise 2.E.1 — Solution

**Strategy:** Start from an unvisited node, run BFS to find all reachable nodes. Repeat until all nodes visited.

# Mini-Exercise 2.E.1 — Solution

**Strategy:** Start from an unvisited node, run BFS to find all reachable nodes. Repeat until all nodes visited.

```
def find_components(graph):
    visited = set()
    components = []
    for node in graph:
        if node not in visited:
            distances = bfs(graph, node)
            component = set(distances.keys())
            visited |= component
            components.append(component)
    return components

# Test with disconnected graph
graph = {
    "A": ["B"], "B": ["A", "C"], "C": ["B"], # component 1
    "X": ["Y"], "Y": ["X", "Z"], "Z": ["Y"] # component 2
}
print(find_components(graph))
# [{'A', 'B', 'C'}, {'X', 'Y', 'Z'}]
```

# Wrap-Up

- **Networks have properties** that demand formal description
- **Graphs** provide a precise model: nodes, edges, weights, direction, time
- **Paths** give us reachability and distance
- **BFS** is a central traversal algorithm with guaranteed shortest paths
- **Dijkstra's** extends BFS to weighted graphs using a priority queue
- **Connectivity** helps us detect structure and giant components

## Reading

Chapter 2 of Easley & Kleinberg (2010) covers graph theory foundations, paths, BFS, and connectivity in detail.

## Image Credits & References

Easley, David and Kleinberg, Jon (2010). Networks, Crowds, and Markets: Reasoning About a Highly Connected World. *Cambridge University Press*. <https://www.cs.cornell.edu/home/kleinber/net-works-book/>